FreshMart Practice — Student Guide

Oct 11, 2026 · @Abiola

In this practice you repeat both class days on your own laptop. **Day 1:** Python cleans FreshMart's 3 messy sales files and a script runs by itself on a timer. **Day 2:** the same script updates a Google Sheets dashboard and a Power BI dashboard, and n8n emails you when the numbers change or something breaks. Follow the parts in order. Each step tells you what to do, and **Check** tells you what you should see before you move on. Each day takes about 2 hours.

What's in your folder

Unzip `FreshMart_Student_Practice` into your **Documents** folder. Inside you'll find:

File or folder	What it is
`STUDENT_GUIDE.pdf`	This guide
`Day1_Cleaning.ipynb`	Day 1 notebook: cleaning. Cells with `___` are blanks you fill in
`Day2_Dashboards.ipynb`	Day 2 notebook: Google Sheets, Power BI, n8n. Also has blanks
`data/raw/`	The 3 messy branch files: Ikeja, Wuse, Garki
`data/new_week/`	Lekki's first file and a broken file. Leave them there until Part 8
`clean_sales.py`	The cleaning script from class. It fixes the 6 problems
`update_dashboard.py`	Cleans, sends to Google Sheets, then tells n8n how it went
`scheduler.py`	Runs a script again every minute, by itself
`n8n_webhook.txt`	You paste your n8n web address here in Part 7
`FreshMart_n8n_workflow.json`	A ready-made n8n workflow, only if you get stuck in Part 7

File or folder	What it is
`output/`	Empty for now. Reports get saved here

You don't need to edit any `.py` file except one line of `scheduler.py` in Part 8.

Part 1 — Get your laptop ready (10 min)

You need: Anaconda (from class), **Power BI Desktop** (free, Microsoft Store, Windows only), a **personal Gmail** (not work or school), and your **n8n Cloud** login. If your n8n trial has ended, sign up again at n8n.io.

1. Open **Anaconda Prompt** from the Start menu.
2. Type `cd /d` and a space. Drag the `FreshMart_Student_Practice` folder from File Explorer into the black window. Press **Enter**.
3. Install the libraries. Type this and press Enter:

```
pip install pandas openpyxl gspread requests schedule
```

4. Check everything is installed. Type:

```
python -c "import pandas, openpyxl, gspread, requests, schedule; print('All good')"
```

Check: it prints **All good**. If you see an error, look in *If something goes wrong* near the end.

Day 1 — Clean the messy files

Day 1 — Look at the mess first (5 min)

Open `data/raw/ikeja_week41.xlsx` in Excel and scroll slowly. Then open the Wuse file. Try to spot 6 problems before Python shows you:

1. Column names differ between files (`Qty` vs `Quantity`)
2. Branch names are spelled many ways (`ikeja`, `IKEJA`, `Ikj`)
3. Dates are written in different ways (`03/10/2026`, `3 Oct 2026`)
4. Some prices are text (`₦9,500`)
5. Some sales are typed twice
6. Some quantities are empty

Close Excel before you go on. Python can't save over a file that Excel has open.

Day 1 — Work through the notebook (1 hour 45 min)

1. In Anaconda Prompt (inside your practice folder), type `jupyter notebook` and press Enter. A browser tab opens.
2. Click `Day1_Cleaning.ipynb`. Keep the Anaconda Prompt window open the whole time.
3. Run each cell with **Shift + Enter**. Replace every `___` before you run that cell. Read the comment next to each blank: it tells you what goes there.

Notebook step	What you do	Check
Step 0	Run it	`Ready!`
Step 1 — Look at one file	Fill the file path (in quotes) and how many rows to show. Then fill `info` and `value_counts`	A messy table. `info()` shows Qty has missing values. Ikeja is spelled 4 ways
Step 2 — Read all files	Fill the pattern that finds every `.xlsx` in `data/raw`. Then read each file and add it to the list	3 file names. Then 16 column names, which is wrong: it should be 6
Step 3 — Problem 1: column names	Fill the new name for `payment_method`, the method that makes letters small, and the dictionary to rename with	6 columns and `(75, 6)`
Step 4 — Problem 2: branch names	Fill the method that Capitalises Each Word, which word to keep (counting starts at 0), and the right spelling of Ikj	Garki 25, Ikeja 26, Wuse 24
Step 5 — Problem 3: dates	Fill `format` and `dayfirst` (the comment tells you)	7 dates, 2026-10-03 to 2026-10-09

Notebook step	What you do	Check
Step 6 — Problem 4: prices	Fill the pandas function that turns text into numbers	min 350, max 9500
Step 7 — Problem 5: duplicates	Fill the method that finds duplicates, and the one that drops them	4 duplicates, 71 rows left
Step 8 — Problem 6: missing quantities	Fill the method that finds empty values, then the column to check	5 missing rows, then `Clean rows: 66`
Step 9 — Report	Revenue = quantity × unit price. Then group by branch	Ikeja 757,400, Wuse 657,550, Garki 565,300
Step 10 — Save to Excel	Fill today's date variable. Add the 3 missing sheets, copying the `by_branch` line	`Saved output/freshmart_report_....xlsx`. Open it: 5 sheets

Why don't we fill empty quantities with 0? You'd be inventing sales. In real work you send those rows back to the manager, which is why Step 8 saves them to `output/rows_to_check.xlsx`.

Day 1 — Make it run by itself (15 min)

Step 11 — From notebook to script. A notebook is for exploring. A script is a file the computer runs without you. Open `clean_sales.py` in Notepad: it is everything you just did, in one file, with each problem numbered. Then run the Step 11 cell (`!python clean_sales.py`).

Check: the last line says **Done: 66 clean rows, total revenue ₦1,980,250**.

Step 12 — The scheduler. Open a **second** Anaconda Prompt (leave the Jupyter one alone). Type `cd /d` and a space, drag your practice folder in, press Enter. Then type:

```
python scheduler.py
```

Check: it runs once straight away, then again every minute without you touching anything. Open `output/automation_log.txt` in Notepad and see the entries pile up. In real life you'd set it to `schedule.every().monday.at("07:00")`.

Press **Ctrl + C** in that window to stop it. **Day 1 is done.**

Day 2 — Live dashboards and alerts

Day 2 uses the second notebook, `Day2_Dashboards.ipynb`. It re-uses your Day 1 script, so you don't redo the cleaning.

Part 2 — Make your Google robot key (20 min)

Python can't log in with your Gmail password. So you create a robot account, called a *service account*. The robot gets its own email address and a key file. Later you share your Sheet with the robot, like sharing with a colleague. It's free, no card needed.

Already did this in class? If `credentials.json` from class still works, copy it into this folder and skip to Part 3.

1. Go to **console.cloud.google.com** and sign in with your **personal Gmail**. If it asks you to accept terms, tick and click **Agree and continue**.
2. At the top left, click the project box (it may say *Select a project*). Click **New project**. Name it `freshmart-class`. Click **Create**.
3. Wait until it's done. Click the project box again and choose `freshmart-class`. Check the top bar shows `freshmart-class`.
4. Click the search bar at the top. Type `Google Sheets API`. Click it, then click **Enable**.
5. Search again: `Google Drive API`. Click it, then click **Enable**. (Python needs Drive to find your Sheet by its name.)
6. Search: `Service accounts`. Open it. Click **+ Create service account**.
7. Name: `freshmart-bot`. Click **Create and continue**. Skip the next two boxes: click **Continue**, then **Done**.
8. In the list, click the `freshmart-bot@...` email. Open the **Keys** tab. Click **Add key → Create new key → JSON → Create**. A file downloads.
9. Open your Downloads folder. Rename the new file to exactly `credentials.json`. Move it into the `FreshMart_Student_Practice` folder, next to `clean_sales.py`.

Check: `credentials.json` is in your practice folder. **Watch out:** if the downloaded file shows no `.json` ending, Windows is hiding endings. Then rename it to just `credentials`, otherwise it becomes `credentials.json.json`.

Keep it safe: this file is like a password. Never post it in a group, email it, or upload it to GitHub.

Part 3 — Create and share your Google Sheet (5 min)

1. Open `credentials.json` with Notepad (right-click → Open with → Notepad). Find the line `"client_email"`. Copy the email after it. It looks like `freshmart-bot@freshmart-class.iam.gserviceaccount.com`.
2. In your browser, go to **sheets.new**. A blank Google Sheet opens. Click *Untitled spreadsheet* at the top left and name it exactly `FreshMart Dashboard`: capital F, capital D, one space.
3. Click the green **Share** button. Paste the robot's email. Choose **Editor**. Untick **Notify people**. Click **Share**. If Google warns that the address is outside your organisation, click **Share anyway**.
4. In Anaconda Prompt (still inside your practice folder), type:

```
python -c "import gspread;
print(gspread.service_account(filename='credentials.json').open('FreshMart
Dashboard').title)"
```

Check: it prints `FreshMart Dashboard`. You're connected. If it shows an error, look in *If something goes wrong*.

Part 4 — Open the notebook and send data to Google Sheets (25 min)

1. In Anaconda Prompt (inside your practice folder), type `jupyter notebook` and press Enter. A browser tab opens.
2. Click `Day2_Dashboards.ipynb`.
3. Keep this Anaconda Prompt window open the whole time. Jupyter runs inside it.

Now work through the notebook. To run a cell, click inside it and press **Shift + Enter**. Replace every `___` with the right code before you run that cell.

Notebook step	What you do	Check
Step 0	Run it	`Ready!`
Step 1 — Re-use the cleaning	Fill 1 blank: which table goes into `summarise()`	3 branches, Ikeja on top (757,400)
Step 2 — Connect	Fill 2 blanks: your key file name and your Sheet name, both in quotes	`Connected to: FreshMart Dashboard` + a link. Open the link
Step 3 — Send clean data	Fill 1 blank: the method that turns column names into a list. Then run the `get_tab` cell	Your Sheet has a *Clean Data* tab with 66 rows
Step 4 — Send summaries	Replace the 3 `___` lines with 3 `send_table(...)` lines, copying the *By Branch* line	Tabs *By Branch, By Product, By Day, By Payment*

Try each blank yourself first. Look at the line above it and the comment next to it. Still stuck after 5 minutes? See *Answers to the blanks* at the end.

What just happened: the charts you build next read these tabs. Python rewrites the tabs. So the charts update by themselves.

Part 5 — Build 3 charts in Google Sheets (10 min)

You build the charts by hand, once. After that, Python keeps them up to date.

1. Open the **By Branch** tab. Drag from **A1 down to B10**. Click **Insert → Chart**. On the right, under *Chart type*, choose **Column chart**.
2. Open **By Day**. Select **A1 to B32**. Click **Insert → Chart**, choose **Line chart**.
3. Open **By Payment**. Select **A1 to B10**. Click **Insert → Chart**, choose **Pie chart**.

Why select more rows than you have? Today there are 3 branches. If a 4th branch comes, it lands in row 5, and your chart already covers it. You'll see this in Part 8.

Go back to the notebook, **Step 5**.

Part 6 — Power BI dashboard (20 min)

Windows only. On a Mac, skip to Part 7.

1. In the notebook, run the **Step 5** cell (`!python clean_sales.py`). **Check:** it ends with **Done: 66 clean rows**.
2. Open **Power BI Desktop**. Close the welcome screen. Click **Blank report** if it asks.
3. Click **Get data → Excel workbook**. Go to your practice folder, open `output`, and choose `freshmart_clean_latest.xlsx`.
4. In the Navigator window, tick **clean_data** only. Click **Load**.
5. **Card:** in the **Visualizations** pane on the right, click the **Card** icon. From the **Data** pane, drag `revenue` onto it. **Check:** about **1.98M**.
6. **Bar chart:** click an empty spot on the white page. Click **Clustered bar chart**. Drag `branch` to the **Y-axis** box and `revenue` to the **X-axis** box.
7. **Line chart:** click an empty spot. Click **Line chart**. Drag `date` to **X-axis** and `revenue` to **Y-axis**. If you see Year / Quarter / Month, click the small arrow next to `date` in the X-axis box and choose **date** instead of *Date Hierarchy*.
8. **Donut chart:** click an empty spot. Click **Donut chart**. Drag `payment` to **Legend** and `revenue` to **Values**.
9. Click **File → Save as**. Save it as `FreshMart.pbix` in your practice folder.

Why `freshmart_clean_latest.xlsx`? The script saves a dated report every run, plus this one "latest" file that it always overwrites. So Power BI always knows where to look.

Part 7 — n8n email alerts (25 min)

n8n Cloud lives on the internet, so it can't open files on your laptop. But your laptop can **send n8n a message** at a special web address called a **webhook**. The path: Python on your laptop → webhook → n8n Cloud → Gmail.

A. Catch the message from Python

1. Log in to **n8n Cloud**. Click **Create workflow**. Click the name at the top and call it `FreshMart Alerts`.
2. Click **Add first step**, search **Webhook**, and choose it. Set **HTTP Method** to `POST`. Set **Path** to `freshmart`.
3. At the top of the Webhook panel, make sure **Test URL** is selected. Click the URL to copy it. It contains `/webhook-test/`.
4. In your practice folder, open `n8n_webhook.txt` with Notepad. Delete everything, paste the URL, and press **Ctrl + S**.

5. Back in n8n, click **Listen for test event**. Then in the notebook, run the **Step 6** cell (`!python update_dashboard.py`).

Check: the notebook shows `Told n8n: success (n8n replied 200)`. In n8n, the Webhook shows your data under `body`: `status: success`, `total_revenue_text: ₦1,980,250`, `top_branch: Ikeja`. This is exactly what your Python sent.

B. Decide: did it work?

6. Close the Webhook panel. Click the **+** after the Webhook and add an **If** node.
7. For the first value, drag `status` from the left panel into the box. It shows `{{ $json.body.status }}`. In the middle, choose **String → is equal to**. In the second value, type `success`.

C. Send the emails

8. On the **true** output of the If node, click **+**, search **Gmail**, and choose **Send a message**.
9. Under *Credential*, click **Create new credential → Sign in with Google**. Choose your Gmail and click **Allow**.
10. Fill in the Gmail node:
    - **To:** your own email
    - **Subject:** type `FreshMart dashboard updated -` then drag in `total_revenue_text`
    - **Email Type:** Text
    - **Message:** type labels and drag in fields: Branches: `branches`, Total: `total_revenue_text`, Top branch: `top_branch`, Rows to check: `rows_to_check`, Dashboard: `sheet_url`
11. On the **false** output, add another **Gmail → Send a message**. Same credential, same To. Subject: `FreshMart script FAILED`. In **Message**, first click **Expression** above the box, then type `Error: {{ $json.body.error }}`. (Without *Expression*, n8n sends the curly brackets as plain text.)

D. Test, then go live

12. In n8n, click **Execute workflow** at the bottom. Then run the notebook **Step 6** cell again.

Check: an email arrives within seconds. Check Spam if you don't see it.

13. Click **Publish** at the top right. Open the Webhook node, switch to **Production URL** (it has `/webhook/`, no `-test`), copy it, paste it into `n8n_webhook.txt` in place of the old one, and save.

Don't skip step 13. The Test URL only works while you click *Listen*. The Production URL works all the time once the workflow is published.

Stuck? In n8n, open the **⋯ menu → Import from file** and choose `FreshMart_n8n_workflow.json` from your folder. In both Gmail nodes, pick your credential and replace `PUT-YOUR-EMAIL-HERE` with your email. Then do step 13.

Part 8 — The magic moment, then a broken file (15 min)

Start the scheduler

1. Open `scheduler.py` with Notepad. Right now it runs the cleaning script. Switch it to the dashboard script: change the two lines so they look exactly like this, then press **Ctrl + S**:

```
# from clean_sales import main as job
from update_dashboard import scheduled_run as job
```

2. Open a **second** Anaconda Prompt (leave the Jupyter one alone). Type `cd /d` and a space, drag your practice folder in, press Enter. Then type:

```
python scheduler.py
```

Check: it prints `Scheduler started...` and runs once. **No email comes, and that's correct.** The scheduler runs every minute, but only messages you when the numbers change or something breaks.

3. Put your Google Sheet (*By Branch* tab), your email and Power BI side by side on your screen.

A new branch arrives

FreshMart just opened a branch in **Lekki**. Its file is messy too: `LEKKI`, `Lekki Phase 1`, prices with **₦**.

4. In the notebook, run the **Step 7** cell. It copies Lekki's file into `data/raw`. Then take your hands off the keyboard and wait 1 minute.

Check, without pressing anything:

- **Scheduler window:** the script reads 4 files.
- **Google Sheet:** the bar chart shows **4 bars, Lekki in second place** (₦694,700). *Last updated* in cell D1 changes.
- **Email:** *FreshMart dashboard updated - ₦2,674,950*.
- **Power BI:** click **Home → Refresh**. Lekki appears there too.

You changed **zero** lines of code. `Lekki Phase 1` became `Lekki` by itself, because of the "keep the first word" rule in `clean_sales.py`. Good cleaning rules work on data you haven't seen yet.

When things break, you get told

5. Keep the scheduler running. In the notebook, run the **Step 8** cell. It adds a damaged file.

Check, within 1 minute: an email **FreshMart script FAILED** with the error *Excel file format cannot be determined*. n8n took the If node's false path.

6. Run the next notebook cell to remove the broken file. The next run works again. No new email comes, because the numbers didn't change.
7. Click the scheduler window and press **Ctrl + C** to stop it.

You did it. Messy Excel files → Python cleans them on a timer → Excel report + Google Sheet → Power BI, and Python → n8n → email. Put this on your CV: *Automated weekly sales reporting pipeline with Python, Google Sheets API, Power BI and n8n alerts.*

Practise again from the start

1. In `Day2_Dashboards.ipynb`, run the last cell, **Practise again from the start**. It removes Lekki and the broken file from `data/raw`.
2. Open `scheduler.py` and switch the two lines back: remove the `#` in front of `from clean_sales import main as job`, and put a `#` in front of `from update_dashboard import scheduled_run as job`. Save.
3. In Jupyter, click **Kernel → Restart & Clear Output**, and start again from Step 0 of either notebook. Day 1 can be repeated as often as you like. Your Google key, Sheet, Power BI file and n8n workflow keep working, so you can skip Parts 2, 3 and 7 next time.

If something goes wrong

Read the **last line** of the red error, find it below, and do the fix.

Error contains	What it means	Fix
`No module named 'gspread'` (or `requests`, `schedule`)	Library not installed	In the notebook run `!pip install gspread requests schedule`, then **Kernel → Restart**
`FileNotFoundError: ... credentials.json`	Key file missing, misnamed, or in the wrong folder	Name must be exactly `credentials.json` (not `.json.json`), next to `clean_sales.py`
`FileNotFoundError: ... data/raw` or `No Excel files found`	Jupyter was opened from the wrong folder	Close Jupyter. In Anaconda Prompt: `cd /d` + drag the practice folder in, then `jupyter notebook`
`SpreadsheetNotFound`	Sheet name is wrong, or not shared with the robot	Name must be exactly `FreshMart Dashboard`. Share it with the `client_email` as **Editor**
`has not been used in project` or `PERMISSION_DENIED`	Sheets API or Drive API not enabled	Part 2, steps 4–5: click **Enable** on both. Wait 2 minutes, try again
Can't create a key ("key creation is disabled")	You used a work or school Google account	Use a personal Gmail and do Part 2 again
`PermissionError: [Errno 13]`	That Excel file is open in Excel	Close it in Excel, run again
`NameError: name 'sales' is not defined`	Cells run out of order	Click **Run → Run All Above Selected Cell**
`'python' is not recognized`	You used normal Command Prompt	Use **Anaconda Prompt** from the Start menu
`No n8n webhook URL in n8n_webhook.txt`	URL not pasted, or file not saved	Paste the URL as the only line, press **Ctrl + S**

Error contains	What it means	Fix
`n8n replied 404`	Test URL used without *Listen*, or workflow not published	Click **Listen for test event**, or **Publish** and use the Production URL
n8n gets the data but no email	Gmail credential missing, or email in Spam	Pick the credential in the Gmail node. Check Spam. Look at n8n's **Executions** for a red error
Email shows `{{ $json.body.error }}` as text	Field not set to Expression	Click **Expression** above the Message box
No email while the scheduler runs	Nothing changed, so no alert (on purpose)	Normal. Running `python update_dashboard.py` directly always sends one
`ValueError: ... format='mixed'`	Your pandas is too old (below 2.0)	Run !pip install --upgrade pandas in the notebook, then Kernel → Restart
`KeyError: 'branch'`	Day 1 Step 3 (fix column names) was skipped	Run the Step 3 cell again, then carry on
`BadZipFile` or a file named `~$ikeja...`	A branch file is open in Excel	Close Excel and run the cell again

Numbers don't match this guide? Check Lekki isn't still in `data/raw`. With 3 branches: 66 rows, ₦1,980,250. With Lekki: 90 rows, ₦2,674,950.

Still stuck? Post a screenshot of the full error in the class group. Never post `credentials.json`.

Answers to the blanks

Only look after you've tried.

Day 1 (`Day1_Cleaning.ipynb`)

```
# Step 1
ikeja = pd.read_excel("data/raw/ikeja_week41.xlsx")
ikeja.head(10)
ikeja.info()
ikeja["Branch"].value_counts()

# Step 2
files = sorted(glob.glob("data/raw/*.xlsx"))
    df = pd.read_excel(f)
    frames.append(df)

# Step 3
    "payment_method": "payment",
    df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
    df = df.rename(columns=column_names)

# Step 4
sales["branch"] = sales["branch"].str.strip().str.title()
sales["branch"] = sales["branch"].str.split().str[0]
sales["branch"] = sales["branch"].replace({"Ikj": "Ikeja"})

# Step 5
sales["date"] = pd.to_datetime(sales["date"], format="mixed", dayfirst=True)

# Step 6
sales["unit_price"] = pd.to_numeric(sales["unit_price"])

# Step 7
print("Duplicate rows:", sales.duplicated().sum())
sales = sales.drop_duplicates()

# Step 8
missing = sales[sales["quantity"].isna()]
sales = sales.dropna(subset=["quantity"])

# Step 9
sales["revenue"] = sales["quantity"] * sales["unit_price"]
by_branch = sales.groupby("branch")
["revenue"].sum().sort_values(ascending=False).reset_index()

# Step 10
```

```
report_file = f"output/freshmart_report_{today}.xlsx"
    by_product.to_excel(writer, sheet_name="by_product", index=False)
    by_day.to_excel(writer, sheet_name="by_day", index=False)
    by_payment.to_excel(writer, sheet_name="by_payment", index=False)
```

Day 2 (`Day2_Dashboards.ipynb`)

Step 1:

```
summaries = summarise(sales)
```

Step 2:

```
gc = gspread.service_account(filename="credentials.json")
sheet = gc.open("FreshMart Dashboard")
```

Step 3:

```
rows = [sales.columns.tolist()] + sales.astype(str).values.tolist()
```

Step 4 (the three `___` lines):

```
send_table(sheet, "By Product", summaries["by_product"])
send_table(sheet, "By Day", summaries["by_day"])
send_table(sheet, "By Payment", summaries["by_payment"])
```